

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY



SOICT

FINAL PROJECT REPORT

Topic: **Parallelization Methods for CNN-Based Computer Vision Inference**

Course: **Parallel and Distributed Programming**

Course code: IT4130E

Lecturer: Nguyen Tuan Dung

Group members: Luu Hieu An - 202400093

Nguyen Phu An - 20235466

Pham Chi Bang - 20235477

Nguyen Thanh Lam - 20235518

Hanoi, 2026

Contents

Abstract	1
1 Introduction	2
2 Problem Description	3
2.1 Problem Context	3
2.2 YOLO Video People Counting	3
2.3 Temporal-Spatial Task Definition	4
2.4 Post-Processing Challenge	4
2.5 Parallel-Programming Focus	5
2.6 Correctness Targets	5
3 Parallelization Methods	6
3.1 Task-Parallel YOLO Inference	6
3.1.1 Frame and Tile Tasks	6
3.1.2 Mapping Mode 1: Rank Mapping	6
3.1.3 Mapping Mode 2: Weighted-Node Mapping	6
3.1.4 Communication Strategy and Topology	6
3.1.5 Bounding Box Remapping and Global Merge	7
3.1.6 Parallel Algorithm	7
3.2 Metrics and Load Balance	7
4 Experimental Setup	9
4.1 Physical Cluster	9
4.2 Software and Data	9
4.3 Metrics	9
5 Results and Discussion	10
5.1 Detector Accuracy Under Tile Splitting	10
5.2 Input-Size Selection	10
5.3 Granularity and Load Balance	11
5.4 Speedup and Efficiency	12
5.5 Weighted Mapping	13
5.6 Non-Blocking Gather Note	13
5.7 YOLO Parallel Correctness	13
5.8 Summary of Findings	14
6 Conclusion	15

Abstract

Video people counting is an important computer vision problem for crowd monitoring, traffic analysis, safety management, and real-time camera systems, but applying a CNN detector to every frame of a video is computationally expensive. This report presents *MPI-Based YOLO People Counting*, a C++17/OpenMPI pipeline that accelerates pretrained YOLO inference by treating video analysis as a task-parallel workload. The input video is decomposed temporally into frames and spatially into image tiles, so each frame-tile pair becomes an independent MPI task assigned to a rank for local YOLO inference. After inference, rank 0 gathers the detected boxes, remaps tile-local coordinates back to the original frame, removes duplicate detections near tile boundaries, applies global non-maximum suppression, and produces frame-level people counts. The report studies this pipeline as a distributed-memory parallel program, focusing on decomposition, task mapping, communication, load balancing, and the distinction between preserving the serial pipeline output and measuring detector accuracy against ground truth. Overall, the work shows how a practical video people-counting application can be expressed as a task-level MPI problem while exposing the main trade-offs between parallel speedup, communication overhead, tile granularity, and heterogeneous machine capacity.

1 Introduction

Video people counting is a practical computer vision problem with applications in crowd monitoring, traffic analysis, safety management, and camera-based automation. A modern detector such as YOLO can identify people in individual images, but applying it repeatedly to every frame of a video quickly becomes computationally expensive [5, 6]. This makes the problem useful for a parallel-programming study: it is realistic enough to require a full inference pipeline, yet structured enough to expose clear opportunities for decomposition, scheduling, communication analysis, and speedup measurement.

This project focuses on inference rather than model training. The detector itself is a pretrained YOLO model, while the parallel-programming work is the C++17/OpenMPI runtime built around it [4, 2]. The system decomposes a video temporally into frames and spatially into image tiles, then treats each frame-tile pair as an MPI task. Each rank runs YOLO on its assigned tasks through a long-lived local worker process. After inference, rank 0 gathers detections, converts tile-local bounding boxes back to full-frame coordinates, removes duplicate detections near tile boundaries, computes frame-level people counts, and writes the experiment artifacts.

The report evaluates the implementation as a distributed-memory parallel program and separates parallel correctness from detector accuracy. Parallel correctness asks whether the MPI implementation preserves the output of the serial version of the same pipeline, while detector accuracy asks whether YOLO’s predicted counts match human ground truth from MOT17-derived data [3]. This distinction is important because the detector may undercount people in crowded scenes even when the MPI program is correct. The experiments therefore measure correctness, input-size suitability, scheduling behavior, granularity, speedup, and heterogeneous rank placement.

The main contributions and insights of this technical report are:

1. It formulates YOLO video people counting as a task-parallel MPI workload over frames and image tiles.
2. It presents an OpenMPI implementation that combines rank-based mapping, node-weighted mapping, rank-local YOLO workers, duplicate removal, and metric collection.
3. It verifies that the MPI pipeline preserves serial output in the correctness experiment, while also showing that detector accuracy against ground truth is a separate limitation.
4. It analyzes speedup and remaining bottlenecks, especially communication overhead, tile granularity, master-side post-processing, and unequal machine capacity.

2 Problem Description

2.1 Problem Context

Video people counting asks a system to estimate how many people appear in each frame of a video. This problem is useful in crowd monitoring, classroom analytics, transportation observation, safety systems, and other camera-based applications where the number of people changes over time. In this report, people counting is implemented as an object-detection problem: a detector predicts person bounding boxes, and the final count for a frame is the number of accepted person boxes after post-processing.

The computational challenge comes from repeated CNN inference. A single YOLO inference may be manageable, but a video contains hundreds or thousands of frames, and each frame may be split into several regions to expose more parallel work. The total workload therefore grows with both the number of frames and the number of tiles per frame. This makes the application a good parallel-programming problem: it contains many independent inference calls, but the final counting step still requires careful global merging.

The project focuses on inference rather than training. The YOLO model is fixed and pretrained; the report does not modify model weights or propose a new detector. The technical problem is to build a distributed-memory runtime that executes the same inference pipeline faster on multiple MPI ranks while preserving the output of the corresponding serial pipeline.

2.2 YOLO Video People Counting

The input is a finite video sequence:

$$V = \{F_1, F_2, \dots, F_N\}. \quad (2.1)$$

For each frame F_i , the detector produces a set of candidate person detections. Each detection contains a bounding box, a confidence score, and a class label. After filtering and duplicate removal, the target output is the number of final person boxes:

$$\text{count}(F_i) = |\{b \mid b \text{ is a final person bounding box in } F_i\}|. \quad (2.2)$$

The full output of the pipeline is therefore a sequence of frame-level counts:

$$C = \{\text{count}(F_1), \text{count}(F_2), \dots, \text{count}(F_N)\}. \quad (2.3)$$

The detector backend is a pretrained YOLO11n model executed through a local Python worker process [6]. YOLO11 follows the usual one-stage detector structure with a feature-extraction backbone, a feature-fusion neck, and multi-scale detection heads, as shown in Figure 2.1. In this project, YOLO is treated as a detector backend. The MPI program does not split individual neural-network layers or kernels; instead, it distributes repeated YOLO calls across independent video regions.

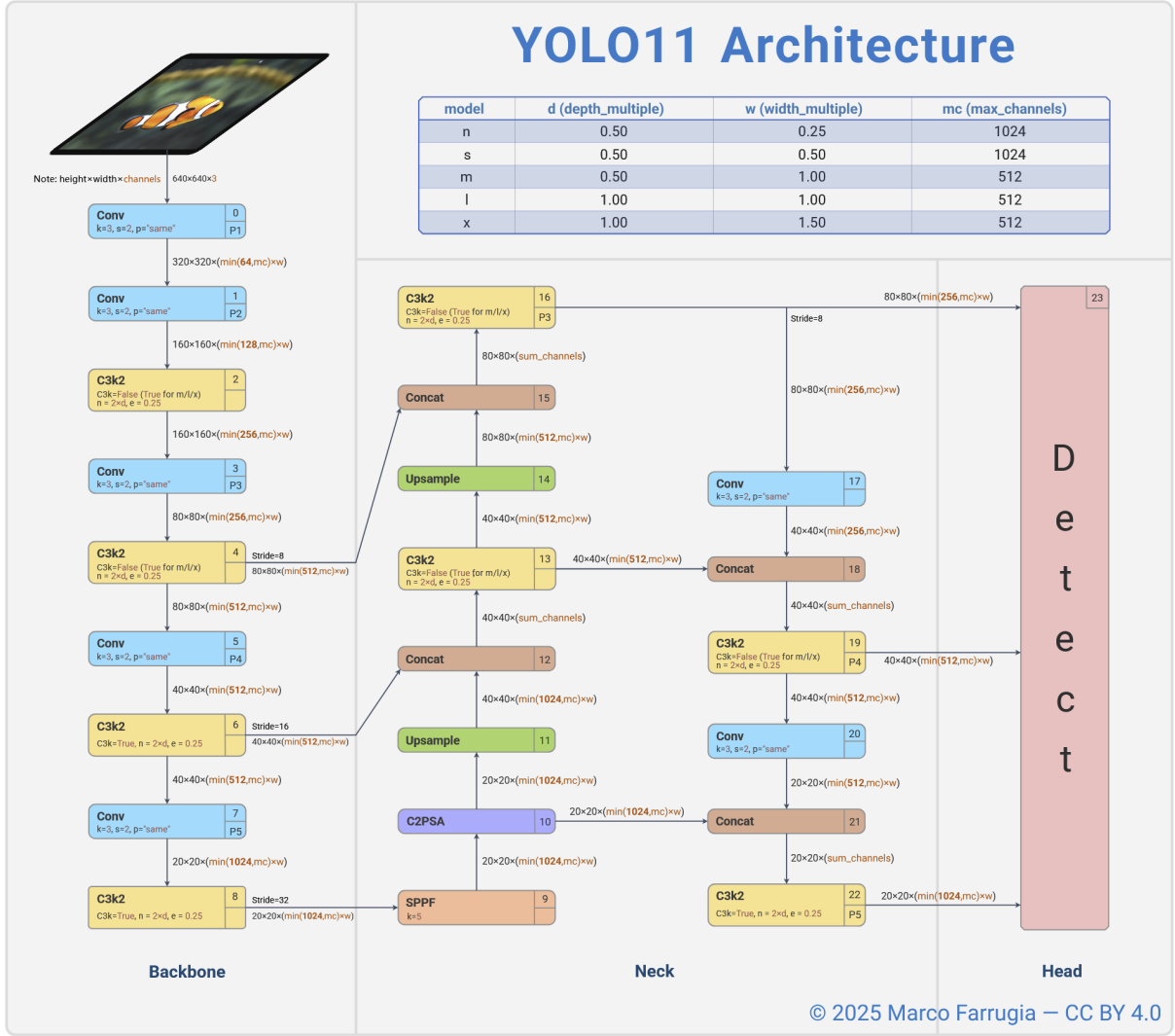


Figure 2.1: YOLO11 object-detection architecture used as the detector backend in this project. Source: Farrugia, CC BY 4.0, Zenodo DOI 10.5281/zenodo.17508018 [1].

2.3 Temporal-Spatial Task Definition

Frame-only parallelism is possible because frames are naturally ordered but mostly independent for this counting task. However, if only complete frames are distributed, the number of tasks may be too small to keep many MPI ranks busy, especially for short videos or small experiments. The project therefore uses a hybrid decomposition: each video is split by time into frames, and each frame is split by space into image tiles.

For a tile grid with R rows and C columns, frame F_i is decomposed into:

$$F_i \rightarrow \{T_{i,1}, T_{i,2}, \dots, T_{i,R \times C}\}. \quad (2.4)$$

A parallel task is one YOLO inference on one frame-tile pair:

$$task(i, j) = T_{i,j}, \quad |\mathcal{T}| = N \times R \times C. \quad (2.5)$$

This definition turns video inference into a task-parallel workload. Larger values of R and C create more tasks, which can improve utilization and load balancing. At the same time, finer tiling can reduce detector context, cut people near tile boundaries, create duplicate boxes, and increase the amount of data that must be gathered by rank 0. The problem is therefore not only to create more tasks, but also to choose a granularity that exposes parallelism without making merging and communication too expensive.

2.4 Post-Processing Challenge

The output of each tile is local to that tile. If a detected box has local coordinates (x_1, y_1, x_2, y_2) , these coordinates must be shifted back into the coordinate system of the original frame before the final count can be computed. For a tile whose top-left corner is (x_s, y_s) , the global coordinates are:

$$x^{global} = x^{local} + x_s, \quad y^{global} = y^{local} + y_s. \quad (2.6)$$

This coordinate remapping is necessary but not sufficient. A person standing near a tile boundary may be detected in more than one tile. If all tile detections were counted directly, the final frame count could be too high. Therefore, rank 0 must merge detections from all tiles of the same frame, apply tile ownership filtering, and run global non-maximum suppression before producing the final count:

$$B_i^{final} = NMS(B_i^{raw}, \theta), \quad count(F_i) = |B_i^{final}|. \quad (2.7)$$

This post-processing step is the main reason the problem is not purely embarrassingly parallel. Detector calls can be distributed across ranks, but final counting requires global information about all detections belonging to the same frame.

2.5 Parallel-Programming Focus

The central parallel-programming question is how to execute the frame-tile inference tasks across multiple MPI ranks while preserving the serial pipeline output and measuring the cost of distribution. The implementation uses task parallelism, rank mapping as the baseline, node-weighted mapping for the heterogeneous cluster, and duplicate-aware result gathering.

The YOLO pipeline exercises the following parallel-programming concepts:

Table 2.1: Parallel-programming concepts in the YOLO pipeline

Concept	YOLO MPI implementation
Parallelism level	Task parallelism across repeated YOLO inference calls
Decomposition	Hybrid temporal-spatial decomposition into frame-tile tasks
Mapping	Baseline rank mapping and node-weighted frame mapping
Communication	Direct gather to rank 0, or local merge on each node before rank 0
Load balancing	Tile granularity, node weights, and local rank distribution
Correctness target	MPI frame counts match serial frame counts after the same post-processing pipeline

2.6 Correctness Targets

The report uses two different notions of correctness. The first is parallel correctness. The serial YOLO baseline processes the same frame-tile task list without MPI distribution and applies the same post-processing pipeline. The MPI implementation is considered correct when it produces the same final people count as this serial baseline for each tested frame:

$$\Delta_i = |count_{parallel}(F_i) - count_{serial}(F_i)|. \quad (2.8)$$

The correctness experiment reports mismatched frames, maximum count error, and mean count error. A zero mismatch means the parallelization has preserved the serial pipeline output for the tested configuration.

The second notion is detector accuracy. Detector accuracy compares YOLO’s predicted counts against human ground-truth counts from MOT17-derived data. This is a separate question because a model can undercount people in crowded frames even if the MPI implementation is perfectly consistent with the serial pipeline. The report therefore evaluates model accuracy and parallel correctness separately: one measures the quality of the detector, while the other measures whether the distributed implementation changes the pipeline result.

3 Parallelization Methods

3.1 Task-Parallel YOLO Inference

The program does not split the internal layers of the YOLO neural network. Instead, it parallelizes the repeated inference calls produced by video processing. Each MPI rank receives a subset of frame-tile tasks, runs YOLO locally, and returns person detections for global merging. Therefore, the level of parallelism is **task parallelism**.

The decomposition technique is a **hybrid temporal-spatial task decomposition**. The temporal dimension decomposes the video into frames, while the spatial dimension decomposes each frame into image tiles. This is not a recursive or speculative decomposition; each task is an independent YOLO inference call on one tile of one frame.

3.1.1 Frame and Tile Tasks

Let the input video contain N frames. Each frame can be split into an $R \times C$ grid of tiles. The number of inference tasks is

$$|\mathcal{T}| = N \times R \times C.$$

Each task is one YOLO inference on one tile of one frame. A small grid such as 2×2 preserves more visual context and usually gives detector behavior closer to the original-frame baseline. A larger grid such as 5×4 creates more parallel tasks, but it also creates more tile boundaries, more duplicated detections, and more work for the final merge stage.

3.1.2 Mapping Mode 1: Rank Mapping

The first mapping mode is `--mapping rank`. All frame-tile tasks are flattened into a one-dimensional task list. For a frame id f and tile id t , where each frame has $T = R \times C$ tiles, the task id is

$$q = fT + t.$$

With P MPI ranks, tasks are assigned in a round-robin/block-cyclic style:

$$\text{owner}(q) = q \bmod P.$$

This mapping is simple and has low scheduling overhead. However, tiles from the same frame may be processed on different physical machines. As a result, many raw bounding boxes must be transferred to rank 0, and rank 0 must perform most of the final merging work.

3.1.3 Mapping Mode 2: Weighted-Node Mapping

The second mapping mode is `--mapping weighted-node`. This mode is used to better match the heterogeneous three-machine cluster. Instead of distributing every tile directly across all ranks, complete frames are first assigned to physical machines according to machine weights. In the final configuration, the selected weights are

$$\text{master} : \text{node1} : \text{node2} = 4 : 6 : 2.$$

This means node1 receives the largest share of frames, the master receives a medium share, and node2 receives the smallest share. For every group of 12 frames, frames 0–3 are assigned to the master, frames 4–9 to node1, and frames 10–11 to node2. The pattern then repeats.

After a frame is assigned to a physical machine, only the ranks on that machine divide the tiles of that frame among themselves. This mapping has two advantages. First, faster machines receive more work. Second, most duplicated boxes from tiles of the same frame can be merged locally before crossing the network.

3.1.4 Communication Strategy and Topology

The communication pattern depends on the mapping mode.

With `--mapping rank`, the topology is a **centralized master-worker gather**. Worker ranks run local YOLO inference and send serialized detections and timing metrics to rank 0. Rank 0 converts tile-local coordinates back to full-frame coordinates, merges detections from all tiles, removes duplicates, applies global non-maximum suppression, and computes the final people count.

With `--mapping weighted-node`, the topology is **hierarchical**. Ranks on the same physical machine first send their detections to a node leader. The node leader performs a local merge for frames owned by that

machine. Then only the smaller merged result is sent to rank 0. Rank 0 still performs the final global merge, but it receives fewer raw detections.

The implementation supports both blocking and non-blocking result transfer. The stable benchmark numbers reported in this report use one consistent benchmark set. The non-blocking gather branch is treated as an implementation optimization: it changes how messages are transferred, but it does not change YOLO inference, coordinate remapping, duplicate removal, or final counting.

3.1.5 Bounding Box Remapping and Global Merge

Each YOLO task produces person bounding boxes in tile-local coordinates. If a tile starts at position (x_s, y_s) in the original frame, a detected box coordinate is converted as

$$x^{global} = x^{local} + x_s, \quad y^{global} = y^{local} + y_s.$$

After coordinate conversion, detections from all tiles belonging to the same frame are merged. A person near a tile boundary may appear in two neighboring tiles, so duplicate removal is required. The final detections for frame F_i are computed as

$$B_i^{final} = NMS(B_i^{raw}, \theta), \quad count(F_i) = |B_i^{final}|.$$

This global merge step is the reason the problem is not purely embarrassingly parallel. YOLO inference tasks are independent, but correct frame-level counting requires all detections belonging to the same frame.

3.1.6 Parallel Algorithm

Algorithm 1 MPI YOLO People-Counting Pipeline

- 1: Split the input video into frames.
 - 2: Split each frame into an $R \times C$ grid of image tiles.
 - 3: Build the list of frame-tile inference tasks.
 - 4: Assign tasks using either rank mapping or weighted-node mapping.
 - 5: **for** each MPI rank in parallel **do**
 - 6: Select the tasks assigned to this rank.
 - 7: **for** each assigned frame-tile task **do**
 - 8: Crop the tile from the original frame.
 - 9: Run YOLO inference on the tile.
 - 10: Keep only person-class detections.
 - 11: Store bounding boxes and timing metrics.
 - 12: **end for**
 - 13: Send local detections for merging.
 - 14: **end for**
 - 15: **if** rank is a node leader **then**
 - 16: Merge detections inside the physical node.
 - 17: Send compact node-level detections to rank 0.
 - 18: **end if**
 - 19: **if** rank is 0 **then**
 - 20: **for** each frame **do**
 - 21: Convert tile-local boxes to global frame coordinates.
 - 22: Merge boxes from all tiles of the frame.
 - 23: Remove duplicated boxes near tile boundaries.
 - 24: Apply global non-maximum suppression.
 - 25: Count the remaining boxes as the number of people.
 - 26: **end for**
 - 27: Write per-frame counts and runtime metrics.
 - 28: **end if**
-

3.2 Metrics and Load Balance

The report records wall-clock runtime, compute-oriented runtime, per-rank compute time, communication time, and idle time. Wall-clock runtime is the main runtime used for input-size selection and speedup.

Speedup is computed as

$$S_p = \frac{T_1}{T_p},$$

where T_1 is the one-process wall-clock runtime and T_p is the wall-clock runtime using p MPI processes. Parallel efficiency is

$$E_p = \frac{S_p}{p}.$$

Load balance is measured with the idle-gap indicator

$$idle_{gap} = \frac{T_{\max} - T_{\min}}{T_{\max}},$$

where $T_{\max}$ and $T_{\min}$ are the maximum and minimum total rank times. A smaller idle gap means the ranks finish more evenly. In this project, the weighted 4/6/2 configuration is used because it reduces idle gap on the heterogeneous three-machine cluster.

4 Experimental Setup

4.1 Physical Cluster

The project uses three physical MacBook machines connected through a local network. The benchmark mode uses CPU execution to match the course emphasis on MPI processes and processor assignment. GPU/MPS execution is kept for demonstration and live-camera use rather than formal CPU speedup measurement.

Table 4.1: Physical cluster roles

Host	Machine type	Role in experiments
master	MacBook Air M4	Coordinates execution, gathers results, stores outputs, and can run local tasks.
node1	MacBook Pro M4	Worker host; measured results suggest it can sustain more assigned ranks.
node2	MacBook Air M2	Worker host; weighted placement assigns fewer ranks to reduce imbalance.

4.2 Software and Data

The implementation uses C++17, OpenMPI [4, 2], Python helper scripts, and a pretrained Ultralytics YOLO detector [6]. The main dataset source is MOT17-derived video data [3]. Compact MOT17 subsets are used for quick correctness and granularity experiments; longer sequence segments are used for the required input-size and speedup runs.

Table 4.2: Main YOLO experimental configuration

Item	Setting
Detector	Pretrained YOLO11n, person-class filtering
Parallel framework	C++17 and OpenMPI
Benchmark device	CPU
Dataset	MOT17-derived sequences
Main process count	12 MPI processes
Main long-run grid	5x4 regions
Scheduler	Static block-cyclic task assignment
Placement variants	Uniform 4/4/4 and weighted 3/6/3, 4/6/2

4.3 Metrics

The report uses the following metrics from the project reports and generated CSV design:

- serial-versus-MPI correctness: mismatched frames and count error;
- detector accuracy: MAE, RMSE, percentage error, exact-match ratio;
- runtime with communication: wall-clock time including coordination;
- compute-oriented run time: comparison runtime reported by the scripts with communication-related reporting minimized;
- speedup and efficiency: $S_p = T_1/T_p$, $E_p = S_p/p$;
- idle-gap indicator for load-balance analysis.

5 Results and Discussion

5.1 Detector Accuracy Under Tile Splitting

This experiment checks whether splitting each frame into spatial tiles changes YOLO detector accuracy against MOT17 ground-truth counts. It evaluates detector behavior under tiling, not parallel speedup or latency.

Table 5.1: YOLO accuracy comparison across baseline and tile-splitting configurations

Configuration	Grid	Frames	Runtime (s)	MAE	RMSE	Percent err.	Pred. avg.
Single-machine baseline	1x1	300	86.4	7.983	8.269	0.490	8.223
MPI tiled	2x2	300	31.0	7.983	8.269	0.490	8.223
MPI tiled	4x3	300	37.0	8.223	8.517	0.504	7.977
MPI tiled	5x4	300	54.764	8.383	8.682	0.514	7.812

Table 5.1 reports the official 300-frame runtimes for the baseline and tile-splitting configurations. The 1x1 baseline is not tile-parallel, so it cannot use the extra MPI work units created by spatial splitting and is slower than the 4x3 MPI tiled run despite having fewer detector calls. Among the MPI tiled configurations, runtime generally increases from 2x2 to 4x3 to 5x4 because each frame creates more tile tasks and rank 0 must merge more intermediate detections. This increase is not linear in the number of tiles: 2x2 uses fewer tasks, but each tile has a larger resolution and therefore the per-tile inference cost remains significant. The 2x2 configuration has the same MAE, RMSE, percent error, and predicted average count as the single-machine baseline, meaning moderate tiling preserves detector behavior on this subset. Finer grids slightly degrade accuracy: MAE rises to 8.223 for 4x3 and 8.383 for 5x4, while the predicted average count falls from 8.223 to 7.812. This suggests that aggressive tiling introduces border effects, reduced image context, and imperfect duplicate handling near tile boundaries.

5.2 Input-Size Selection

This experiment selects the main workload size required by the course. Here, “Frames” means the number of video frames processed in one run, not the number of videos. Since each frame is split into tiles, the actual number of YOLO inference tasks is

$$tasks = frames \times R \times C.$$

The compact quick tests use a 4×3 grid, while the longer benchmark runs use the final 5×4 grid. Therefore, this table should be interpreted as an input-size selection experiment rather than a perfectly controlled scaling experiment across only one parameter.

Table 5.2: Input-size selection for the YOLO MPI benchmark

Frames	Grid	Tasks	Runtime (s)	Time/task (s)	P
150	4x3	1,800	41.8	0.0232	12
300	4x3	3,600	63.5	0.0176	12
600	5x4	12,000	123.667	0.0103	12

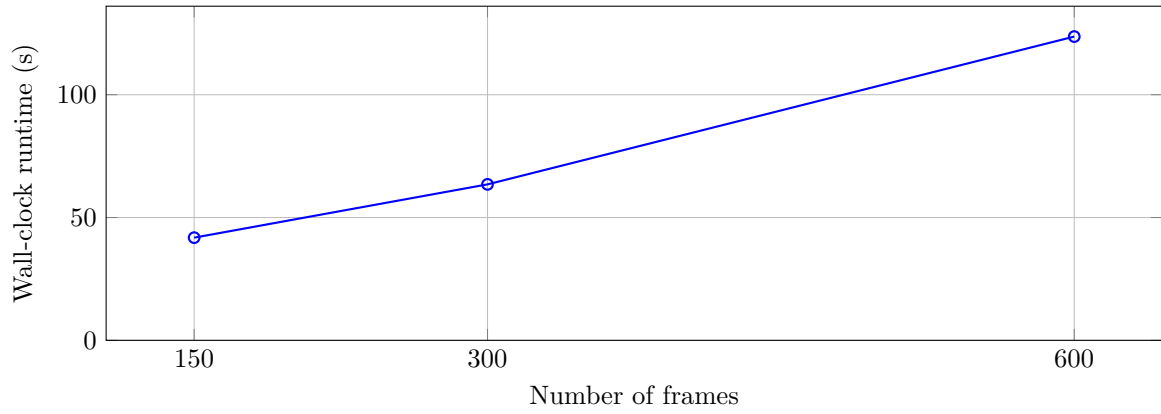


Figure 5.1: Runtime grows with the number of frames and tile tasks. The 600-frame run is selected because it satisfies the required 120–180s interval.

The time-per-task column shows that very small inputs have high overhead because startup, synchronization, and aggregation are amortized over too few tasks. The selected workload is

$$N = 600 \text{ frames}, \quad R \times C = 5 \times 4, \quad P = 12.$$

This produces 12,000 frame-tile inference tasks and a measured wall-clock runtime of 123.667 seconds. Therefore, it satisfies the required 120–180 second interval. The 1200-frame workload is reserved for the strong-scaling experiment in Section 5.4, where the same weighted placement is used and the process count is varied.

5.3 Granularity and Load Balance

This experiment evaluates the effect of tile granularity and process placement. The selected course workload remains $N = 600$ frames as determined in Section 5.2. To compare several tile grids under the same placement without making the report excessively long, this subsection uses a shorter 150-frame diagnostic run. The purpose is to observe how changing the number of tiles per frame affects task count, communication cost, and load balance. A full $N = 600$ per-rank timing run with the final 5×4 grid follows the same trend and is used as the main benchmark for the selected workload.

Table 5.3: Granularity and communication overhead under weighted 4/6/2 placement

Grid	Tasks	Compute (s)	Comm. (s)	Idle gap
1x1	150	18.3	10.674	0.466
2x2	600	25.4	23.5	0.312
4x3	1,800	32.6	37.2	0.253
5x4	3,000	38.1	51.790	0.235

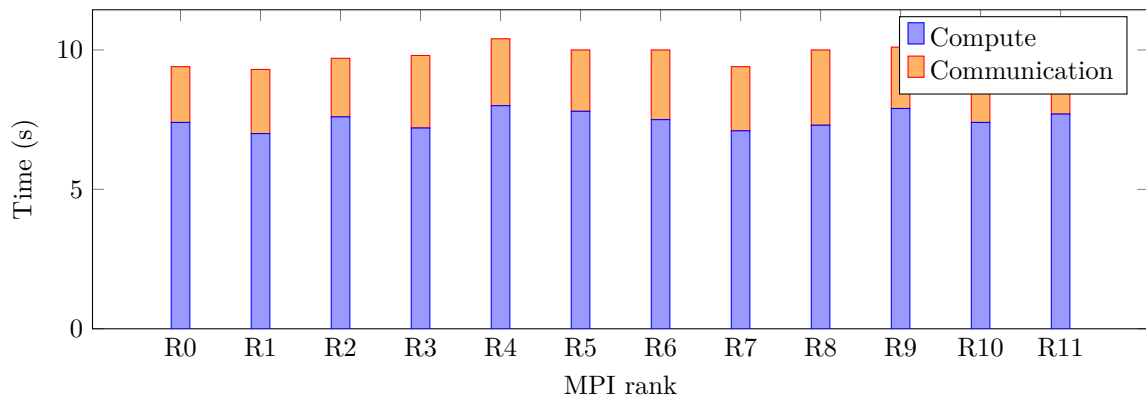


Figure 5.2: Per-rank compute and communication time for the 150-frame diagnostic run under weighted placement.

Because the diagnostic run uses 150 frames, increasing the grid from 1×1 to 5×4 increases the number of frame-tile tasks from

$$150 \times 1 = 150$$

to

$$150 \times 20 = 3000.$$

This gives the scheduler more work units, but it also increases the amount of intermediate detection data that must be transferred and merged. In the measured results, communication time rises from 10.674 seconds at 1×1 to 51.790 seconds at 5×4 . This confirms that finer granularity is not automatically faster: it improves task availability but also increases merging and communication overhead.

The idle-gap indicator remains 0.235 under the weighted 4/6/2 placement, which is below the 25% imbalance threshold. Therefore, the selected weighted placement is considered balanced for this experiment. The per-rank stacked bar chart further shows why load balance should be evaluated at process level rather than only through total runtime: ranks may spend different amounts of time in compute and communication, but the final finishing times remain close enough for the workload to be acceptable.

5.4 Speedup and Efficiency

The speedup experiment uses the selected input size doubled to

$$2N = 1200 \text{ frames.}$$

All multi-rank runs use the weighted 4/6/2 cluster configuration so that the process distribution matches the heterogeneous machines: node1 receives the largest share of work, the master receives a medium share, and node2 receives the smallest share.

Table 5.4: Strong-scaling results for the YOLO MPI pipeline

Processes P	Runtime with comm. (s)	Runtime w/o comm. (s)	Speedup S_p	Efficiency E_p
1	345.677	342.467	1.000	1.000
2	276.155	272.546	1.252	0.626
4	244.747	240.905	1.412	0.353
8	200.770	194.282	1.722	0.215
12	178.306	170.255	1.939	0.162

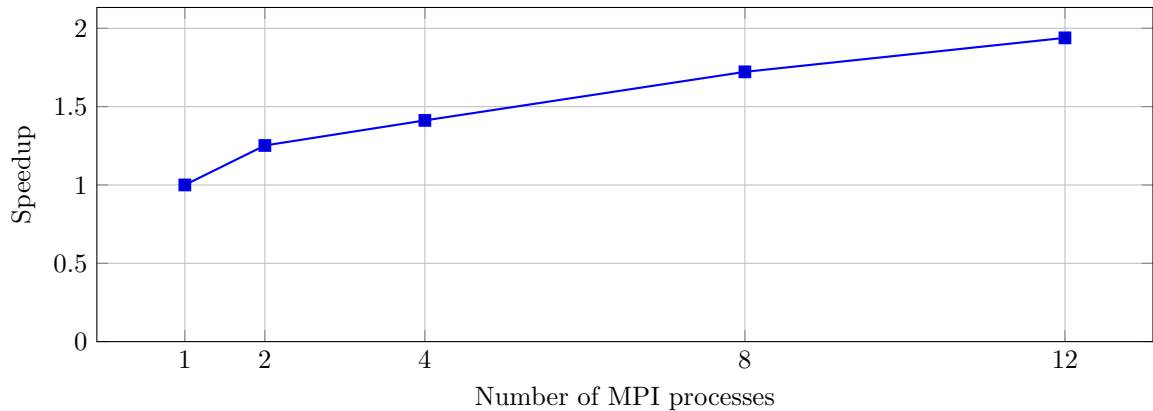


Figure 5.3: Wall-clock speedup grows sublinearly as more MPI ranks are used.

The measured wall-clock speedup rises from 1.252x at $P = 2$ to 1.939x at $P = 12$. This shows that the MPI pipeline benefits from distributing frame-tile YOLO inference across multiple machines. However, the speedup is sublinear. Efficiency decreases as P increases because each additional process contributes less marginal benefit once communication, rank-0 aggregation, duplicate removal, heterogeneous node speeds, and tile-boundary overhead are included. The compute-oriented runtime curve shows the same trend but remains slightly better because it excludes the communication and coordination overhead that is present in the wall-clock metric.

The parallel cost confirms the same trend. Using the standard cost model

$$C_p = pT_p,$$

where T_p is the wall-clock runtime including communication, the two-process run costs

$$2 \times 276.155 = 552.310$$

process-seconds, while the twelve-process run costs

$$12 \times 178.306 = 2139.672$$

process-seconds. Compared with the serial cost $C_1 = 345.677$, the overhead grows from 206.633 process-seconds at $P = 2$ to 1793.995 process-seconds at $P = 12$.

Therefore, the implementation is useful for reducing wall-clock time, but it is not cost-optimal at high process counts. In this workload, $P = 12$ is justified by the course requirement and by the 1.939x wall-clock speedup, while the cost analysis explains why the efficiency falls to 0.162.

5.5 Weighted Mapping

This experiment tests whether equal work placement is appropriate on the heterogeneous three-MacBook cluster. The best measured weighted configuration assigns weight four to the master, six to node1, and two to node2:

$$master : node1 : node2 = 4 : 6 : 2.$$

This gives the strongest machine the largest work share and the slowest machine the smallest work share.

Table 5.5: Placement strategies and measured imbalance

Placement	Master share	Node1 share	Idle gap
Uniform 4/4/4	4	4	0.466
Weighted 3/6/3	3	6	0.312
Weighted 4/6/2	4	6	0.235

Uniform 4/4/4 placement has an idle gap of 0.466 and fails the 0.25 balance criterion. Weighted 3/6/3 improves runtime and idle gap, but it is still not balanced. The best measured configuration is weighted 4/6/2, which reduces the idle gap to 0.235 and satisfies the balance threshold.

This result supports the main mapping claim of the project: equal work shares are not necessarily fair when the physical machines have different throughput. Weighted-node mapping improves both runtime and balance by assigning more frames to faster machines and fewer frames to slower machines.

5.6 Non-Blocking Gather Note

The implementation also contains a two-phase non-blocking gather branch using `MPI_Isend`, `MPI_Irecv`, and `MPI_Waitall`. This branch preserves the same YOLO inference, coordinate-remapping, duplicate-removal, and counting logic. It only changes how serialized result payloads are transferred between ranks.

In this report, the non-blocking branch is treated as an implementation optimization rather than the source of the final rubric numbers. The selected input size, final speedup, and load-balance conclusions are based on one consistent benchmark set from Tables 5.2, 5.4, and 5.5:

$$N = 600 \text{ frames runs in } 123.667 \text{ seconds,}$$

and the $2N = 1200$ -frame speedup reaches

$$S_{12} = 1.939$$

at $P = 12$, while the compute-oriented estimate is approximately

$$S_{12}^{\text{compute}} \approx \frac{342.467}{170.255} \approx 2.011.$$

This separation avoids mixing results from different benchmark branches. The non-blocking gather is still useful as an engineering improvement, but the report uses the stable benchmark set above for the required input-size, granularity, and speedup analysis.

5.7 YOLO Parallel Correctness

This experiment verifies parallel correctness: after task assignment, MPI gathering, coordinate remapping, duplicate removal, and final counting, the MPI output should match the serial pipeline output.

Table 5.6: YOLO serial-versus-MPI correctness

Result	Schedule	Grid	P	Frames	Mismatch	Max err.	Mean err.
Passed	Static	4x3	12	30	0	0	0.000

The MPI run has zero mismatched frames over the 30-frame verification sequence. Both the maximum count error and mean count error are zero, so the parallel pipeline exactly reproduces the serial pipeline for this

tested configuration. This proves serial-versus-MPI consistency; it does not prove that YOLO is accurate against human ground truth, which is measured separately in the detector accuracy section.

5.8 Summary of Findings

The measured YOLO results support the main task-parallel narrative. The implementation preserves serial output, reaches the required input-size runtime, shows measurable speedup, and demonstrates that task granularity and weighted mapping strongly affect performance. The strongest measured improvement in load balance comes from the 4/6/2 weighted configuration, which reduces the idle-gap indicator from 0.466 to 0.235.

6 Conclusion

This project implemented and evaluated a task-parallel people-counting pipeline using YOLO, C++17, and OpenMPI on a three-machine physical cluster. The input video is decomposed both temporally and spatially: frames create independent work over time, while image tiles create finer-grained work inside each frame. These frame-tile tasks are assigned through rank-based mapping and weighted-node mapping, processed locally by YOLO workers, and merged through coordinate remapping, tile-owner filtering, and global duplicate removal. The project demonstrates the main parallel-programming concepts required by the course. The level of parallelism is task parallelism. The decomposition technique is hybrid temporal-spatial decomposition. The mapping strategy includes a simple rank-based mapping and a weighted-node mapping designed for a heterogeneous cluster. The communication pattern is either a centralized gather to rank 0 or a hierarchical node-local merge followed by a compact global gather. Load balance is measured with an idle-gap indicator, and the best measured placement is the weighted 4/6/2 configuration.

The experiments show that the parallel implementation preserves the behavior of the serial tiled pipeline. In the correctness experiment, the MPI run has zero mismatched frames, zero maximum count error, and zero mean count error compared with the serial pipeline. This verifies parallel correctness, although detector accuracy against human ground truth remains a separate limitation of the pretrained YOLO model.

For input-size selection, the selected course workload is

$$N = 600 \text{ frames}$$

with a 5×4 tile grid and 12 MPI processes. This produces 12,000 frame-tile inference tasks and a wall-clock runtime of 123.667 seconds, satisfying the required 120–180 second interval. For the strong-scaling experiment, the workload is doubled to

$$2N = 1200 \text{ frames.}$$

The final measured wall-clock speedup is 1.939x at $P = 12$, while the compute-oriented curve stays slightly better at about 2.011x. The result is useful but sublinear, mainly because of communication overhead, rank-0 aggregation, duplicate removal, tile-boundary effects, and heterogeneous node speeds.

The granularity and mapping experiments show that finer tiling creates more parallel work units but also increases communication and post-processing cost. Therefore, more tasks do not automatically produce better performance. The weighted 4/6/2 placement improves load balance by reducing the idle-gap indicator from 0.466 under uniform 4/4/4 placement to 0.235, which is below the 25% imbalance threshold.

The project also exposes practical limitations. Centralized merging at the master can become a bottleneck, tiled inference can reduce visual context near region boundaries, and the pretrained detector may undercount crowded scenes. Even with these limitations, the system provides a clear demonstration of distributed computer-vision inference with MPI, including decomposition, mapping, communication, load balancing, correctness verification, granularity analysis, and speedup measurement.

Bibliography

- [1] Marco Farrugia. Yolo11 architecture diagram: Visual representation of the yolo11 object detection model. <https://doi.org/10.5281/zenodo.17508018>, 2025.
- [2] Message Passing Interface Forum. Mpi: A message-passing interface standard. <https://www.mpi-forum.org/docs/>, 2023.
- [3] MOTChallenge. Mot17 benchmark. <https://motchallenge.net/data/MOT17/>, 2026.
- [4] Open MPI Project. Open mpi documentation. <https://docs.open-mpi.org/>, 2026.
- [5] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [6] Ultralytics. Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2026.